

```

File Edit Options Buffers Tools Gdb Complete In/Out Signals Help
p p+ Run Continue Next Line Step Line Up Stack Down Stack
9 Type "show configuration" for configuration details.
10 For bug reporting instructions, please see:
11 <http://www.gnu.org/software/gdb/bugs/>.
12 Find the GDB manual and other documentation resources online at:
13 <http://www.gnu.org/software/gdb/documentation/>.
14 For help, type "help".
15 Type "apropos word" to search for commands related to "word"...
16 Reading symbols from a.out...done.
17 (gdb) break main
18 Breakpoint 1 at 0x7c3: file buggyFactorial.cpp, line 13.
19 (gdb) r
20 Starting program: /media/kartha/New Volume1/Vineeth_Projects/OSFY/Gud/a.out
21
22 Breakpoint 1, main () at buggyFactorial.cpp:13
23 13      int result = factorial(5);
24 (gdb)
* 1.1k 1: *gud-a.out* Debugger:run [breakpoint-hit] company 24 : 6 Bottom
1 #include<iostream>
2 #include <assert.h>
3
4 int factorial(int num) {
5     int product = 1;
6     while(num-->0) {
7         product *= num;
8     }
9     return product;
10 }
11
12 int main() {
13     int result = factorial(5);
14     assert(result == 120);
15     return 0;
16 }
- 233 -: buggyFactorial.cpp C++/l company FlyC Abbrev 13 : 0 Top
Switched to thread 1

```

Figure 5: GDB and the code in split windows

```

p p+ Run Continue Next Line Step Line Up Stack
12 Find the GDB manual and other documentation resources online at:
13 <http://www.gnu.org/software/gdb/documentation/>.
14 For help, type "help".
15 Type "apropos word" to search for commands related to "word"...
16 Reading symbols from a.out...done.
17 (gdb) break main
18 Breakpoint 1 at 0x7c3: file buggyFactorial.cpp, line 13.
19 (gdb) r
20 Starting program: /media/kartha/New Volume1/Vineeth_Projects/OSFY/Gud/a.out
21
22 Breakpoint 1, main () at buggyFactorial.cpp:13
23 13      int result = factorial(5);
24 (gdb)
* 1.1k 1: *gud-a.out* Debugger:run [end-stepping-range] company 24 : 6 Bottom
1 #include<iostream>
2 #include <assert.h>
3
4 int factorial(int num) {
5     int product = 1;
6     while(num-->0) {
7         product *= num;
8     }
9     return product;
10 }
11
12 int main() {
13     int result = factorial(5);
14     assert(result == 120);
15     return 0;
16 }
- 233 -: buggyFactorial.cpp C++/l company FlyC Abbrev 5 : 0 Top
Command: -exec-step 1

```

Figure 6: Show the local variables in a separate frame in Emacs

well, you should get the window that's shown in Figure 4.

At the *gdb* prompt, type *break main* and then *r* to run the program. This should start running the program and should break at the *main()*.

As soon as GDB hits the break point at *main*, a new buffer will open up showing the code that you are

debugging. Notice the red dot on the left side, which is where your breakpoint was set. There will be a small indicator that shows which line of the code you are on. Currently, this will be the same as the break point itself (Figure 5).

To debug the factorial function, we need to step into it. For this, you can either use the *gdb* prompt and the *gdb*

command *step*, or you can use the Emacs shortcut *C-c C-s*. There are other similar shortcuts, but I prefer using the GDB commands. So I will use them in the rest of this article.

Let us keep an eye on the local variables while stepping through the factorial number. Check out Figure 6 for how to get an Emacs frame to show the local variables.

Step through the code in the GDB prompt and watch the value of the local variable change. In the first iteration of the loop itself, we see a problem. The value of the product should have been 5 and not 4.

This is where I leave you and now it's up to the readers to explore and discover the magic land called GUD mode. Every *gdb* command works in the GUD mode as well. I leave the fix to this code as an exercise to readers. Explore and see how you can customise things to make your workflow simpler and become more productive while debugging. **END** 🐧

By: Vineeth Kartha

The author is a C++ developer with a passion for open source technologies. When not coding, he can be found chasing bugs (not the software variety) and birds with his camera.